

Chương 4 - Mô hình Huấn luyện

Sổ tay này chứa tất cả các mã mẫu và giải pháp cho các bài tập trong chương 4.

 [Open in Colab](#)  [Open in Kaggle](#)

✓ Thiết lập

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Nó cũng yêu cầu Scikit-Learn $\geq 1.0.1$:

```
from packaging import version
import sklearn

assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

Như chúng ta đã làm trong các chương trước, hãy xác định kích thước phông chữ mặc định để làm cho các hình đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục (nếu nó chưa tồn tại) và xác định chức năng được sử dụng thông qua sổ ghi chép này để lưu các số liệu ở độ phân giải cao cho cuốn sách: `images/training_linear_modelssave_fig()`

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "training_linear_models"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

✓ Hồi quy tuyến tính

✓ Phương trình bình thường

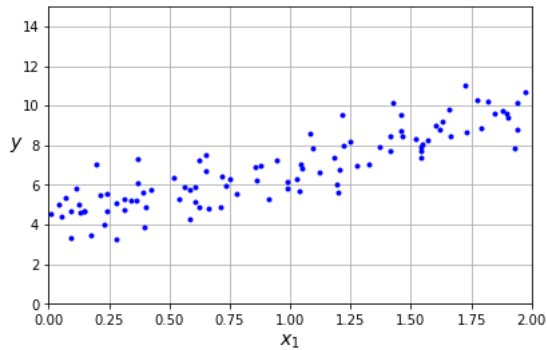
```
import numpy as np

np.random.seed(42) # to make this code example reproducible
m = 100 # number of instances
X = 2 * np.random.rand(m, 1) # column vector
y = 4 + 3 * X + np.random.randn(m, 1) # column vector
```

```
# extra code - generates and saves Figure 4-1

import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(6, 4))
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
save_fig("generated_data_plot")
plt.show()
```



```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

theta_best

```
array([[4.21509616],
       [2.77011339]])
```

```
X_new = np.array([[0], [2]])
X_new_b = add_dummy_feature(X_new) # add x0 = 1 to each instance
y_predict = X_new_b @ theta_best
y_predict
```

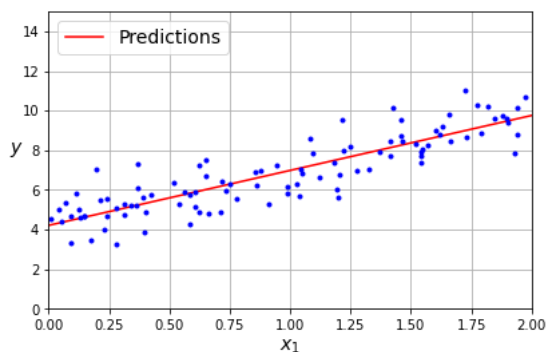
```
array([[4.21509616],
       [9.75532293]])
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4)) # extra code - not needed, just formatting
plt.plot(X_new, y_predict, "r-", label="Predictions")
plt.plot(X, y, "b.")

# extra code - beautifies and saves Figure 4-2
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
plt.legend(loc="upper left")
save_fig("linear_model_predictions_plot")

plt.show()
```



```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = LinearRegression()  
lin_reg.fit(X, y)  
lin_reg.intercept_, lin_reg.coef_
```

```
(array([4.21509616]), array([[2.77011339]]))
```

```
lin_reg.predict(X_new)
```

```
array([[4.21509616],  
       [9.75532293]])
```

Lớp này dựa trên hàm (tên là viết tắt của "bình phương nhỏ nhất"), mà bạn có thể gọi trực tiếp: `LinearRegression.scipy.linalg.lstsq()`

```
theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)  
theta_best_svd
```

```
array([[4.21509616],  
       [2.77011339]])
```

Hàm này tính toán X^+y đầu X^+ là Nghịch đảo giả của X (cụ thể là nghịch đảo Moore-Penrose). Bạn có thể sử dụng `np.linalg.pinv()` Để tính toán trực tiếp Nghịch đảo giả:

```
np.linalg.pinv(X_b) @ y
```

```
array([[4.21509616],  
       [2.77011339]])
```

▼ Xuống dốc gradient

Giảm độ dốc hàng loạt

```
eta = 0.1 # learning rate  
n_epochs = 1000  
m = len(X_b) # number of instances  
  
np.random.seed(42)  
theta = np.random.randn(2, 1) # randomly initialized model parameters  
  
for epoch in range(n_epochs):  
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)  
    theta = theta - eta * gradients
```

Các thông số mô hình được đào tạo:

```
theta
```

```
array([[4.21509616],  
       [2.77011339]])
```

```
# extra code - generates and saves Figure 4-8  
  
import matplotlib as mpl  
  
def plot_gradient_descent(theta, eta):  
    m = len(X_b)  
    plt.plot(X, y, "b.")  
    n_epochs = 1000  
    n_shown = 20  
    theta_path = []  
    for epoch in range(n_epochs):  
        if epoch < n_shown:  
            y_predict = X_new_b @ theta  
            color = mpl.colors.rgb2hex(plt.cm.OrRd(epoch / n_shown + 0.15))  
            plt.plot(X_new, y_predict, linestyle="solid", color=color)  
            gradients = 2 / m * X_b.T @ (X_b @ theta - y)  
            theta = theta - eta * gradients
```

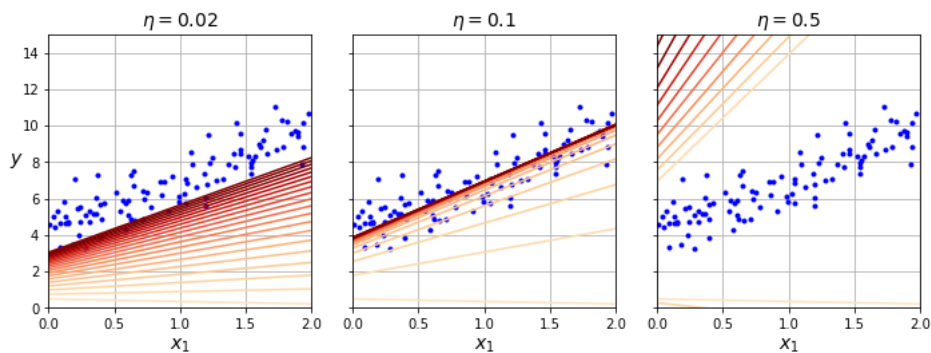
```

        theta_path.append(theta)
    plt.xlabel("$x_1$")
    plt.axis([0, 2, 0, 15])
    plt.grid()
    plt.title(fr"$\eta = \{\eta\}$")
    return theta_path

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

plt.figure(figsize=(10, 4))
plt.subplot(131)
plot_gradient_descent(theta, eta=0.02)
plt.ylabel("$y$", rotation=0)
plt.subplot(132)
theta_path_bgd = plot_gradient_descent(theta, eta=0.1)
plt.gca().axes.yaxis.set_ticklabels([])
plt.subplot(133)
plt.gca().axes.yaxis.set_ticklabels([])
plot_gradient_descent(theta, eta=0.5)
save_fig("gradient_descent_plot")
plt.show()

```



✧ Ngẫu nhiên Gradient Descent

```

theta_path_sgd = [] # extra code - we need to store the path of theta in the
                    # parameter space to plot the next figure

```

```

n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

n_shown = 20 # extra code - just needed to generate the figure below
plt.figure(figsize=(6, 4)) # extra code - not needed, just formatting

for epoch in range(n_epochs):
    for iteration in range(m):

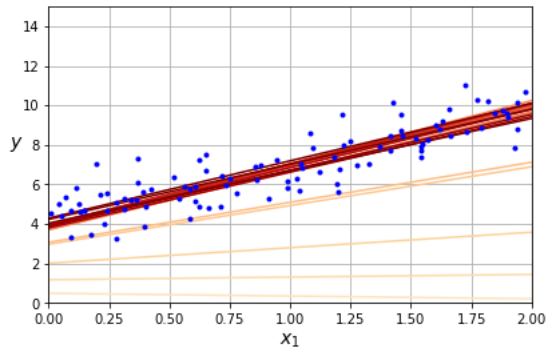
        # extra code - these 4 lines are used to generate the figure
        if epoch == 0 and iteration < n_shown:
            y_predict = X_new_b @ theta
            color = mpl.colors.rgb2hex(plt.cm.OrRd(iteration / n_shown + 0.15))
            plt.plot(X_new, y_predict, color=color)

        random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # for SGD, do not divide by m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
        theta_path_sgd.append(theta) # extra code - to generate the figure

# extra code - this section beautifies and saves Figure 4-10
plt.plot(X, y, "b.")

```

```
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
save_fig("sgd_plot")
plt.show()
```



theta

```
array([[4.21076011],
       [2.74856079]])
```

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
```

```
SGDRegressor(n_iter_no_change=100, penalty=None, random_state=42, tol=1e-05)
```

```
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([4.21278812]), array([2.77270267]))
```

✓ Giảm độ dốc hàng loạt nhỏ

Mã trong phần này được sử dụng để tạo hình tiếp theo, nó không có trong sách.

```
# extra code - this cell generates and saves Figure 4-11

from math import ceil

n_epochs = 50
minibatch_size = 20
n_batches_per_epoch = ceil(m / minibatch_size)

np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

t0, t1 = 200, 1000 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

theta_path_mgd = []
for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(m)
    X_b_shuffled = X_b[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for iteration in range(0, n_batches_per_epoch):
        idx = iteration * minibatch_size
        xi = X_b_shuffled[idx : idx + minibatch_size]
        yi = y_shuffled[idx : idx + minibatch_size]
        gradients = 2 / minibatch_size * xi.T @ (xi @ theta - yi)
        eta = learning_schedule(iteration)
        theta = theta - eta * gradients
```

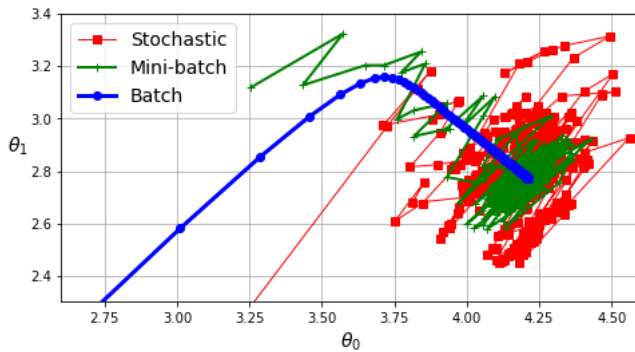
```

theta_path_mgd.append(theta)

theta_path_bgd = np.array(theta_path_bgd)
theta_path_sgd = np.array(theta_path_sgd)
theta_path_mgd = np.array(theta_path_mgd)

plt.figure(figsize=(7, 4))
plt.plot(theta_path_sgd[:, 0], theta_path_sgd[:, 1], "r-s", linewidth=1,
         label="Stochastic")
plt.plot(theta_path_mgd[:, 0], theta_path_mgd[:, 1], "g-+", linewidth=2,
         label="Mini-batch")
plt.plot(theta_path_bgd[:, 0], theta_path_bgd[:, 1], "b-o", linewidth=3,
         label="Batch")
plt.legend(loc="upper left")
plt.xlabel(r"$\theta_0$")
plt.ylabel(r"$\theta_1$", rotation=0)
plt.axis([2.6, 4.6, 2.3, 3.4])
plt.grid()
save_fig("gradient_descent_paths_plot")
plt.show()

```



▽ Hồi quy đa thức

```

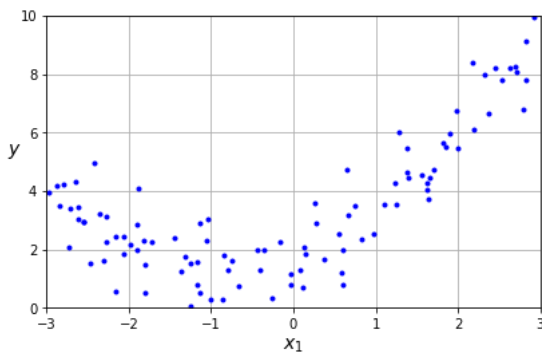
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)

```

```

# extra code - this cell generates and saves Figure 4-12
plt.figure(figsize=(6, 4))
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([-3, 3, 0, 10])
plt.grid()
save_fig("quadratic_data_plot")
plt.show()

```



```

from sklearn.preprocessing import PolynomialFeatures

poly_features = PolynomialFeatures(degree=2, include_bias=False)

```

```
X_poly = poly_features.fit_transform(X)
X[0]
```

```
array([-0.75275929])
```

```
X_poly[0]
```

```
array([-0.75275929,  0.56664654])
```

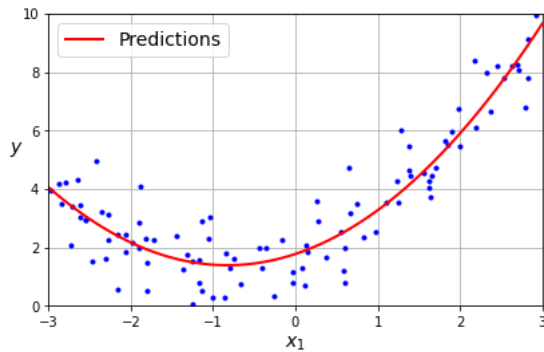
```
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
lin_reg.intercept_, lin_reg.coef_
```

```
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

extra code - this cell generates and saves Figure 4-13

```
X_new = np.linspace(-3, 3, 100).reshape(100, 1)
X_new_poly = poly_features.transform(X_new)
y_new = lin_reg.predict(X_new_poly)

plt.figure(figsize=(6, 4))
plt.plot(X, y, "b.")
plt.plot(X_new, y_new, "r-", linewidth=2, label="Predictions")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.legend(loc="upper left")
plt.axis([-3, 3, 0, 10])
plt.grid()
save_fig("quadratic_predictions_plot")
plt.show()
```



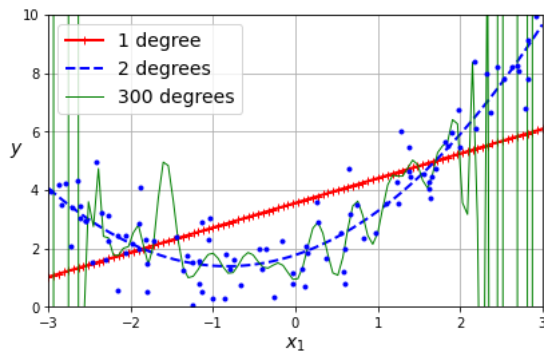
extra code - this cell generates and saves Figure 4-14

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

plt.figure(figsize=(6, 4))

for style, width, degree in (("r+", 2, 1), ("b--", 2, 2), ("g-", 1, 300)):
    polybig_features = PolynomialFeatures(degree=degree, include_bias=False)
    std_scaler = StandardScaler()
    lin_reg = LinearRegression()
    polynomial_regression = make_pipeline(polybig_features, std_scaler, lin_reg)
    polynomial_regression.fit(X, y)
    y_newbig = polynomial_regression.predict(X_new)
    label = f"{degree} degree{'s' if degree > 1 else ''}"
    plt.plot(X_new, y_newbig, style, label=label, linewidth=width)

plt.plot(X, y, "b.", linewidth=3)
plt.legend(loc="upper left")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([-3, 3, 0, 10])
plt.grid()
save_fig("high_degree_polynomials_plot")
plt.show()
```



✓ Đường cong học tập

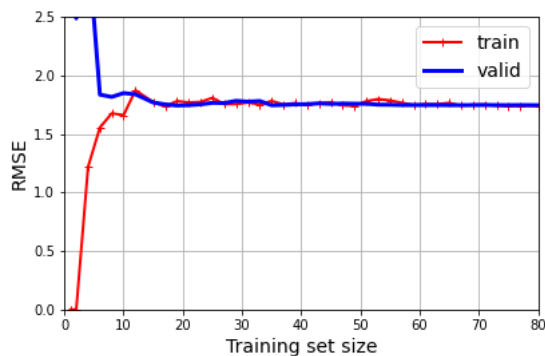
```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.figure(figsize=(6, 4)) # extra code - not needed, just formatting
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")

# extra code - beautifies and saves Figure 4-15
plt.xlabel("Training set size")
plt.ylabel("RMSE")
plt.grid()
plt.legend(loc="upper right")
plt.axis([0, 80, 0, 2.5])
save_fig("underfitting_learning_curves_plot")

plt.show()
```



```
from sklearn.pipeline import make_pipeline

polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

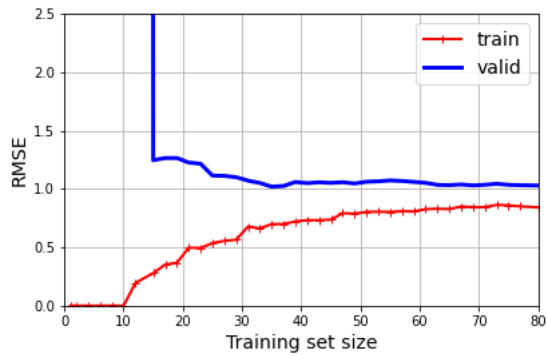
train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")
```

```
# extra code - generates and saves Figure 4-16

train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.figure(figsize=(6, 4))
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
```

```
plt.legend(loc="upper right")
plt.xlabel("Training set size")
plt.ylabel("RMSE")
plt.grid()
plt.axis([0, 80, 0, 2.5])
save_fig("learning_curves_plot")
plt.show()
```



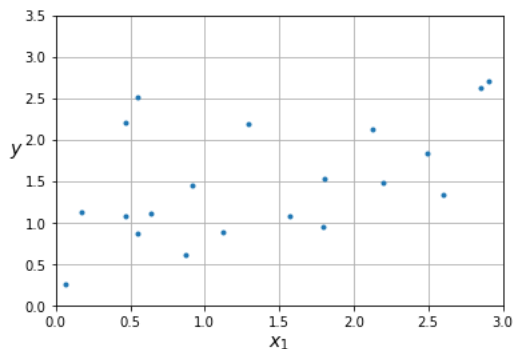
✧ Mô hình tuyến tính chính quy

✧ Hồi quy sườn núi

Hãy tạo một tập dữ liệu tuyến tính rất nhỏ và nhiễu:

```
# extra code - we've done this type of generation several times before
np.random.seed(42)
m = 20
X = 3 * np.random.rand(m, 1)
y = 1 + 0.5 * X + np.random.randn(m, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)
```

```
# extra code - a quick peek at the dataset we just generated
plt.figure(figsize=(6, 4))
plt.plot(X, y, ".")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 3, 0, 3.5])
plt.grid()
plt.show()
```



```
from sklearn.linear_model import Ridge

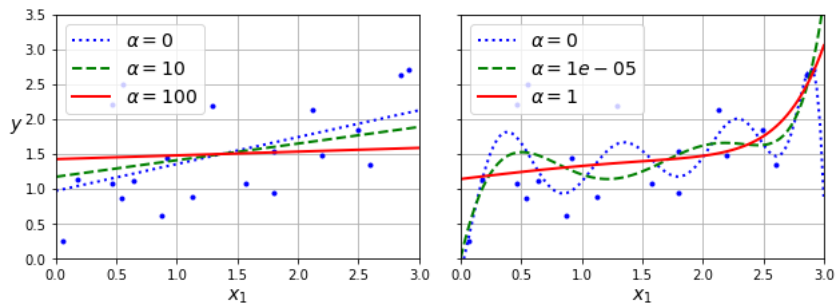
ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
array([[1.55325833]])
```

extra code - this cell generates and saves Figure 4-17

```
def plot_model(model_class, polynomial, alphas, **model_kwargs):
    plt.plot(X, y, "b.", linewidth=3)
    for alpha, style in zip(alphas, ("b:", "g--", "r-")):
        if alpha > 0:
            model = model_class(alpha, **model_kwargs)
        else:
            model = LinearRegression()
        if polynomial:
            model = make_pipeline(
                PolynomialFeatures(degree=10, include_bias=False),
                StandardScaler(),
                model)
        model.fit(X, y)
        y_new_regul = model.predict(X_new)
        plt.plot(X_new, y_new_regul, style, linewidth=2,
                 label=f"$\alpha = {alpha}$")
    plt.legend(loc="upper left")
    plt.xlabel("$x_1$")
    plt.axis([0, 3, 0, 3.5])
    plt.grid()

plt.figure(figsize=(9, 3.5))
plt.subplot(121)
plot_model(Ridge, polynomial=False, alphas=(0, 10, 100), random_state=42)
plt.ylabel("$y$ ", rotation=0)
plt.subplot(122)
plot_model(Ridge, polynomial=True, alphas=(0, 10**-5, 1), random_state=42)
plt.gca().axes.yaxis.set_ticklabels([])
save_fig("ridge_regression_plot")
plt.show()
```



```
sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
                       max_iter=1000, eta0=0.01, random_state=42)
sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
sgd_reg.predict([[1.5]])
```

```
array([1.55302613])
```

```
# extra code - show that we get roughly the same solution as earlier when
# we use Stochastic Average GD (solver="sag")
ridge_reg = Ridge(alpha=0.1, solver="sag", random_state=42)
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
array([[1.55321535]])
```

```
# extra code - shows the closed form solution of Ridge regression,
# compare with the next Ridge model's learned parameters below
alpha = 0.1
A = np.array([[0., 0.], [0., 1.]])
X_b = np.c_[np.ones(m), X]
np.linalg.inv(X_b.T @ X_b + alpha * A) @ X_b.T @ y
```

```
array([[0.97898394],
       [0.3828496 ]])
```

```
ridge_reg.intercept_, ridge_reg.coef_ # extra code
```

```
(array([0.97944909]), array([0.38251084]))
```

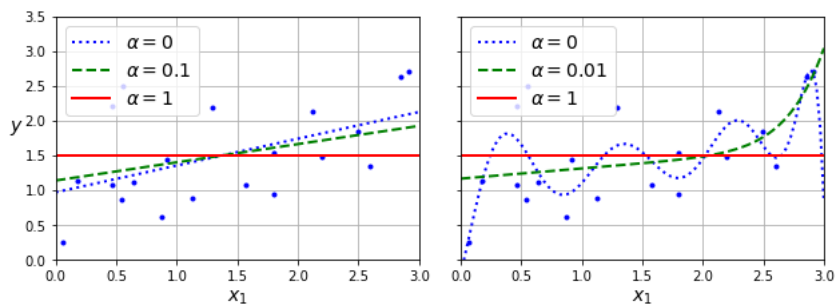
❏ Hồi quy Lasso

```
from sklearn.linear_model import Lasso
```

```
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
lasso_reg.predict([[1.5]])
```

```
array([1.53788174])
```

```
# extra code - this cell generates and saves Figure 4-18
plt.figure(figsize=(9, 3.5))
plt.subplot(121)
plot_model(Lasso, polynomial=False, alphas=(0, 0.1, 1), random_state=42)
plt.ylabel("$y$ ", rotation=0)
plt.subplot(122)
plot_model(Lasso, polynomial=True, alphas=(0, 1e-2, 1), random_state=42)
plt.gca().axes.yaxis.set_ticklabels([])
save_fig("lasso_regression_plot")
plt.show()
```



```
# extra code - this BIG cell generates and saves Figure 4-19
```

```
t1a, t1b, t2a, t2b = -1, 3, -1.5, 1.5
```

```
t1s = np.linspace(t1a, t1b, 500)
t2s = np.linspace(t2a, t2b, 500)
t1, t2 = np.meshgrid(t1s, t2s)
T = np.c_[t1.ravel(), t2.ravel()]
Xr = np.array([[1, 1], [1, -1], [1, 0.5]])
yr = 2 * Xr[:, :1] + 0.5 * Xr[:, 1:]
```

```
J = (1 / len(Xr) * ((T @ Xr.T - yr.T) ** 2).sum(axis=1)).reshape(t1.shape)
```

```
N1 = np.linalg.norm(T, ord=1, axis=1).reshape(t1.shape)
N2 = np.linalg.norm(T, ord=2, axis=1).reshape(t1.shape)
```

```
t_min_idx = np.unravel_index(J.argmin(), J.shape)
t1_min, t2_min = t1[t_min_idx], t2[t_min_idx]
```

```
t_init = np.array([[0.25], [-1]])
```

```
def bgd_path(theta, X, y, l1, l2, core=1, eta=0.05, n_iterations=200):
    path = [theta]
    for iteration in range(n_iterations):
        gradients = (core * 2 / len(X) * X.T @ (X @ theta - y)
                     + l1 * np.sign(theta) + l2 * theta)
        theta = theta - eta * gradients
        path.append(theta)
    return np.array(path)
```

```
fig, axes = plt.subplots(2, 2, sharex=True, sharey=True, figsize=(10.1, 8))
```

```
for i, N, l1, l2, title in ((0, N1, 2.0, 0, "Lasso"), (1, N2, 0, 2.0, "Ridge")):
    JR = J + l1 * N1 + l2 * 0.5 * N2 ** 2
```

```
tr_min_idx = np.unravel_index(JR.argmin(), JR.shape)
t1r_min, t2r_min = t1[tr_min_idx], t2[tr_min_idx]
```

```
levels = np.exp(np.linspace(0, 1, 20)) - 1
```

```

levelsJ = levels * (J.max() - J.min()) + J.min()
levelsJR = levels * (JR.max() - JR.min()) + JR.min()
levelsN = np.linspace(0, N.max(), 10)

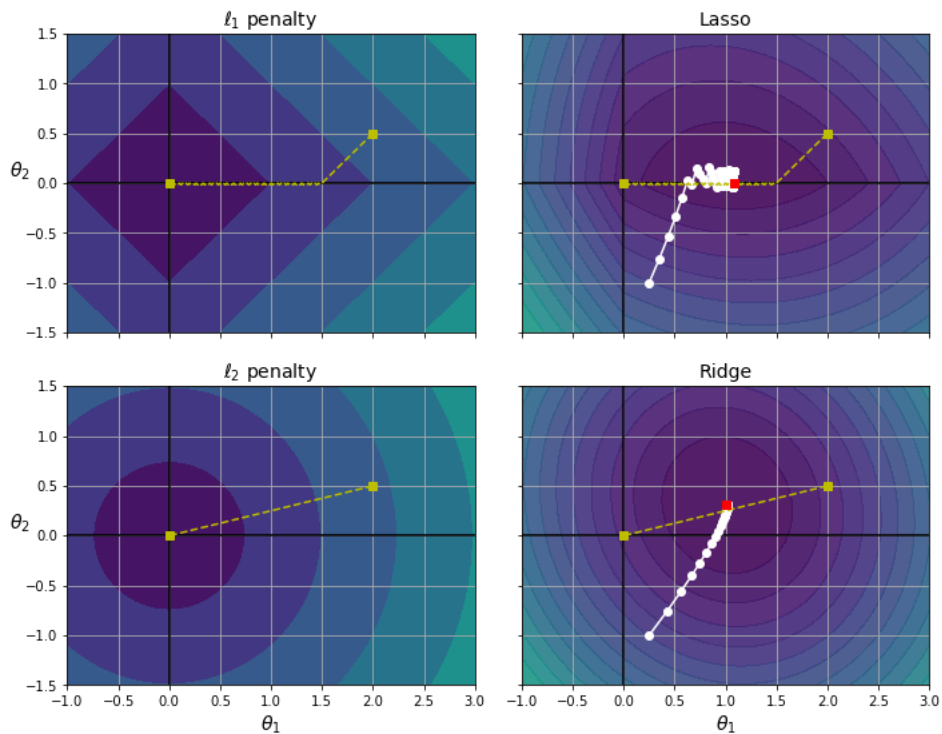
path_J = bgd_path(t_init, Xr, yr, l1=0, l2=0)
path_JR = bgd_path(t_init, Xr, yr, l1, l2)
path_N = bgd_path(theta=np.array([[2.0], [0.5]]), X=Xr, y=yr,
                  l1=np.sign(l1) / 3, l2=np.sign(l2), core=0)

ax = axes[i, 0]
ax.grid()
ax.axhline(y=0, color="k")
ax.axvline(x=0, color="k")
ax.contourf(t1, t2, N / 2.0, levels=levelsN)
ax.plot(path_N[:, 0], path_N[:, 1], "y--")
ax.plot(0, 0, "ys")
ax.plot(t1_min, t2_min, "ys")
ax.set_title(fr"$\ell_{i+1}$ penalty")
ax.axis([t1a, t1b, t2a, t2b])
if i == 1:
    ax.set_xlabel(r"$\theta_1$")
ax.set_ylabel(r"$\theta_2$", rotation=0)

ax = axes[i, 1]
ax.grid()
ax.axhline(y=0, color="k")
ax.axvline(x=0, color="k")
ax.contourf(t1, t2, JR, levels=levelsJR, alpha=0.9)
ax.plot(path_JR[:, 0], path_JR[:, 1], "w-o")
ax.plot(path_N[:, 0], path_N[:, 1], "y--")
ax.plot(0, 0, "ys")
ax.plot(t1_min, t2_min, "ys")
ax.plot(t1r_min, t2r_min, "rs")
ax.set_title(title)
ax.axis([t1a, t1b, t2a, t2b])
if i == 1:
    ax.set_xlabel(r"$\theta_1$")

save_fig("lasso_vs_ridge_plot")
plt.show()

```



↳ Lưới đàn hồi

```
from sklearn.linear_model import ElasticNet
```

```

elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
elastic_net.predict([[1.5]])

```

```
array([1.54333232])
```

▼ Dừng sớm

Hãy quay lại tập dữ liệu bậc hai mà chúng ta đã sử dụng trước đó:

```

from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# extra code - creates the same quadratic dataset as earlier and splits it
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
X_train, y_train = X[: m // 2], y[: m // 2, 0]
X_valid, y_valid = X[m // 2 :], y[m // 2 :, 0]

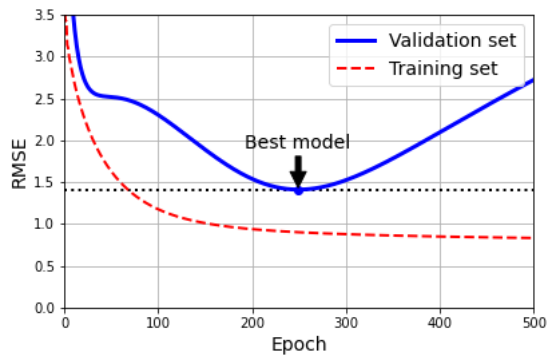
preprocessing = make_pipeline(PolynomialFeatures(degree=90, include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')
train_errors, val_errors = [], [] # extra code - it's for the figure below

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = mean_squared_error(y_valid, y_valid_predict, squared=False)
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)

    # extra code - we evaluate the train error and save it for the figure
    y_train_predict = sgd_reg.predict(X_train_prep)
    train_error = mean_squared_error(y_train, y_train_predict, squared=False)
    val_errors.append(val_error)
    train_errors.append(train_error)

# extra code - this section generates and saves Figure 4-20
best_epoch = np.argmin(val_errors)
plt.figure(figsize=(6, 4))
plt.annotate('Best model',
            xy=(best_epoch, best_valid_rmse),
            xytext=(best_epoch, best_valid_rmse + 0.5),
            ha="center",
            arrowprops=dict(facecolor='black', shrink=0.05))
plt.plot([0, n_epochs], [best_valid_rmse, best_valid_rmse], "k:", linewidth=2)
plt.plot(val_errors, "b-", linewidth=3, label="Validation set")
plt.plot(best_epoch, best_valid_rmse, "bo")
plt.plot(train_errors, "r--", linewidth=2, label="Training set")
plt.legend(loc="upper right")
plt.xlabel("Epoch")
plt.ylabel("RMSE")
plt.axis([0, n_epochs, 0, 3.5])
plt.grid()
save_fig("early_stopping_plot")
plt.show()

```



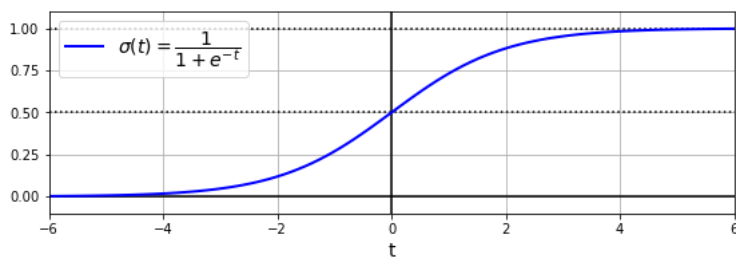
✧ Hồi quy logistic

✧ Ước tính xác suất

extra code - generates and saves Figure 4-21

```
lim = 6
t = np.linspace(-lim, lim, 100)
sig = 1 / (1 + np.exp(-t))

plt.figure(figsize=(8, 3))
plt.plot([-lim, lim], [0, 0], "k-")
plt.plot([-lim, lim], [0.5, 0.5], "k:")
plt.plot([-lim, lim], [1, 1], "k-")
plt.plot([0, 0], [-1.1, 1.1], "k-")
plt.plot(t, sig, "b-", linewidth=2, label=r"$\sigma(t) = \frac{1}{1 + e^{-t}}$")
plt.xlabel("t")
plt.legend(loc="upper left")
plt.axis([-lim, lim, -0.1, 1.1])
plt.gca().set_yticks([0, 0.25, 0.5, 0.75, 1])
plt.grid()
save_fig("logistic_function_plot")
plt.show()
```



✧ Ranh giới quyết định

```
from sklearn.datasets import load_iris
```

```
iris = load_iris(as_frame=True)
list(iris)
```

```
['data',
 'target',
 'frame',
 'target_names',
 'DESCR',
 'feature_names',
 'filename',
 'data_module']
```

```
print(iris.DESCR) # extra code - it's a bit too long
```

```

.. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
  - sepal length in cm
  - sepal width in cm
  - petal length in cm
  - petal width in cm
  - class:
    - Iris-Setosa
    - Iris-Versicolour
    - Iris-Virginica

:Summary Statistics:

=====
      Min   Max   Mean   SD   Class Correlation
=====
sepal length:  4.3   7.9   5.84   0.83    0.7826
sepal width:   2.0   4.4   3.05   0.43   -0.4194
petal length:   1.0   6.9   3.76   1.76    0.9490 (high!)
petal width:   0.1   2.5   1.20   0.76    0.9565 (high!)
=====

:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988

```

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken from Fisher's paper. Note that it's the same as in R, but not as in the UCI Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the pattern recognition literature. Fisher's paper is a classic in the field and is referenced frequently to this day. (See Duda & Hart, for example.) The data set contains 3 classes of 50 instances each, where each class refers to a type of iris plant. One class is linearly separable from the other 2; the latter are NOT linearly separable from each other.

.. topic:: References

- Fisher, R.A. "The use of multiple measurements in taxonomic problems" Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis. (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System Structure and Classification Rule for Recognition in Partially Exposed Environments". IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. PAMI-2, No. 1, 67-71.

```
iris.data.head(3)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

```
iris.target.head(3) # note that the instances are not shuffled
```

```

0    0
1    0
2    0
Name: target, dtype: int64

```

```
iris.target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)

```

```
LogisticRegression(random_state=42)
```

```

X_new = np.linspace(0, 3, 1000).reshape(-1, 1) # reshape to get a column vector
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]

```

```

plt.figure(figsize=(8, 3)) # extra code - not needed, just formatting
plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Not Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
         label="Decision boundary")

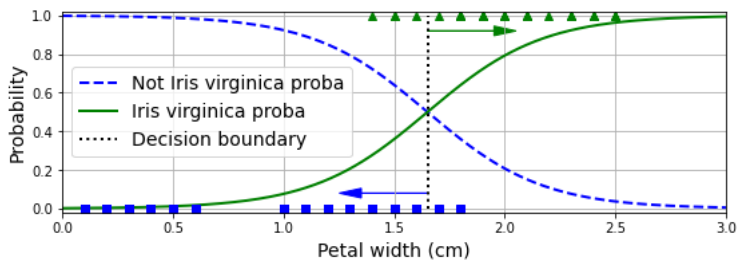
```

```

# extra code - this section beautifies and saves Figure 4-23
plt.arrow(x=decision_boundary, y=0.08, dx=-0.3, dy=0,
         head_width=0.05, head_length=0.1, fc="b", ec="b")
plt.arrow(x=decision_boundary, y=0.92, dx=0.3, dy=0,
         head_width=0.05, head_length=0.1, fc="g", ec="g")
plt.plot(X_train[y_train == 0], y_train[y_train == 0], "bs")
plt.plot(X_train[y_train == 1], y_train[y_train == 1], "g^")
plt.xlabel("Petal width (cm)")
plt.ylabel("Probability")
plt.legend(loc="center left")
plt.axis([0, 3, -0.02, 1.02])
plt.grid()
save_fig("logistic_regression_plot")

```

```
plt.show()
```



```
decision_boundary
```

```
1.6516516516516517
```

```
log_reg.predict([[1.7], [1.5]])
```

```
array([ True, False])
```

```
# extra code - this cell generates and saves Figure 4-24
```

```

X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

```

```

log_reg = LogisticRegression(C=2, random_state=42)
log_reg.fit(X_train, y_train)

```

```

# for the contour plot
x0, x1 = np.meshgrid(np.linspace(2.9, 7, 500).reshape(-1, 1),
                    np.linspace(0.8, 2.7, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()] # one instance per point on the figure
y_proba = log_reg.predict_proba(X_new)
zz = y_proba[:, 1].reshape(x0.shape)

```

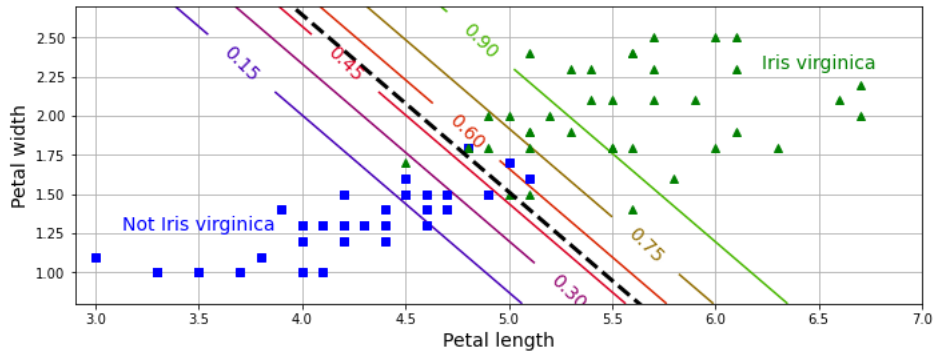
```
# for the decision boundary
```

```

left_right = np.array([2.9, 7])
boundary = -((log_reg.coef_[0, 0] * left_right + log_reg.intercept_[0])
             / log_reg.coef_[0, 1])

plt.figure(figsize=(10, 4))
plt.plot(X_train[y_train == 0, 0], X_train[y_train == 0, 1], "bs")
plt.plot(X_train[y_train == 1, 0], X_train[y_train == 1, 1], "g^")
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg)
plt.clabel(contour, inline=1)
plt.plot(left_right, boundary, "k--", linewidth=3)
plt.text(3.5, 1.27, "Not Iris virginica", color="b", ha="center")
plt.text(6.5, 2.3, "Iris virginica", color="g", ha="center")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.axis([2.9, 7, 0.8, 2.7])
plt.grid()
save_fig("logistic_regression_contour_plot")
plt.show()

```



▼ Hồi quy Softmax

```

X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)

```

```
LogisticRegression(C=30, random_state=42)
```

```
softmax_reg.predict([[5, 2]])
```

```
array([2])
```

```
softmax_reg.predict_proba([[5, 2]]).round(2)
```

```
array([[0. , 0.04, 0.96]])
```

```

# extra code - this cell generates and saves Figure 4-25

from matplotlib.colors import ListedColormap

custom_cmap = ListedColormap(["#fafab0", "#9898ff", "#a0faa0"])

x0, x1 = np.meshgrid(np.linspace(0, 8, 500).reshape(-1, 1),
                     np.linspace(0, 3.5, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()]

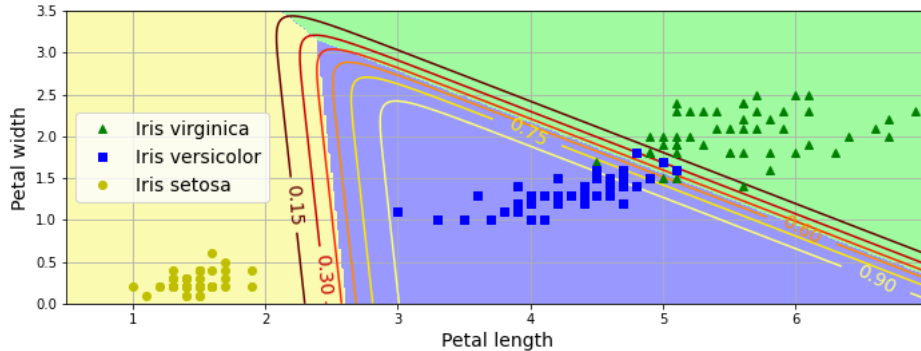
y_proba = softmax_reg.predict_proba(X_new)
y_predict = softmax_reg.predict(X_new)

zz1 = y_proba[:, 1].reshape(x0.shape)
zz = y_predict.reshape(x0.shape)

plt.figure(figsize=(10, 4))
plt.plot(X[y == 2, 0], X[y == 2, 1], "g^", label="Iris virginica")
plt.plot(X[y == 1, 0], X[y == 1, 1], "bs", label="Iris versicolor")
plt.plot(X[y == 0, 0], X[y == 0, 1], "yo", label="Iris setosa")

```

```
plt.contourf(x0, x1, zz, cmap=custom_cmap)
contour = plt.contour(x0, x1, zz1, cmap="hot")
plt.clabel(contour, inline=1)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="center left")
plt.axis([0.5, 7, 0, 3.5])
plt.grid()
save_fig("softmax_regression_contour_plot")
plt.show()
```



✧ Lời giải

✧ Các bài từ 1. tới 11.

1. Nếu bạn có một tập huấn luyện với hàng triệu đặc trưng, bạn có thể sử dụng Thuật toán giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent) hoặc Thuật toán giảm độ dốc theo lô nhỏ (Mini-batch Gradient Descent), và có thể là Thuật toán giảm độ dốc theo lô lớn (Batch Gradient Descent) nếu tập huấn luyện vừa với bộ nhớ. Nhưng bạn không thể sử dụng Phương trình chuẩn (Normal Equation) hoặc phương pháp phân rã SVD (SVD approach) vì độ phức tạp tính toán tăng rất nhanh (hơn bậc hai) theo số lượng đặc trưng.
2. Nếu các đặc trưng trong tập huấn luyện của bạn có thang đo rất khác nhau, hàm chi phí sẽ có hình dạng một cái bát kéo dài, do đó các thuật toán Giảm độ dốc sẽ mất nhiều thời gian để hội tụ. Để giải quyết vấn đề này, bạn nên chuẩn hóa dữ liệu trước khi huấn luyện mô hình. Lưu ý rằng Phương trình chuẩn hoặc phương pháp phân rã SVD sẽ hoạt động tốt mà không cần chuẩn hóa. Hơn nữa, các mô hình có sử dụng điều chuẩn (regularized models) có thể hội tụ đến một giải pháp không tối ưu nếu các đặc trưng không được chuẩn hóa: vì điều chuẩn sẽ phạt các trọng số lớn, các đặc trưng có giá trị nhỏ hơn sẽ có xu hướng bị bỏ qua so với các đặc trưng có giá trị lớn hơn.
3. Thuật toán Giảm độ dốc không thể bị mắc kẹt ở một cực tiểu cục bộ khi huấn luyện một mô hình Hồi quy Logistic vì hàm chi phí của nó là hàm lồi (convex). Lồi có nghĩa là nếu bạn vẽ một đường thẳng giữa hai điểm bất kỳ trên đường cong, đường thẳng đó sẽ không bao giờ cắt đường cong.
4. Nếu bài toán tối ưu là lồi (chẳng hạn như Hồi quy Tuyến tính hoặc Hồi quy Logistic), và giả sử tốc độ học không quá cao, thì tất cả các thuật toán Giảm độ dốc sẽ tiến gần đến điểm tối ưu toàn cục và cuối cùng tạo ra các mô hình khá tương tự nhau. Tuy nhiên, trừ khi bạn giảm tốc độ học dần dần, Thuật toán GD ngẫu nhiên và Thuật toán GD theo lô nhỏ sẽ không bao giờ thực sự hội tụ; thay vào đó, chúng sẽ liên tục nhảy qua lại xung quanh điểm tối ưu toàn cục. Điều này có nghĩa là ngay cả khi bạn cho chúng chạy trong một thời gian rất dài, các thuật toán Giảm độ dốc này sẽ tạo ra các mô hình hơi khác nhau.
5. Nếu lỗi xác thực (validation error) liên tục tăng sau mỗi epoch, thì một khả năng là tốc độ học quá cao và thuật toán đang phân kỳ. Nếu lỗi huấn luyện (training error) cũng tăng, thì đây rõ ràng là vấn đề và bạn nên giảm tốc độ học. Tuy nhiên, nếu lỗi huấn luyện không tăng, thì mô hình của bạn đang quá khớp (overfitting) với tập huấn luyện và bạn nên dừng việc huấn luyện.
6. Do tính chất ngẫu nhiên của chúng, cả Thuật toán giảm độ dốc ngẫu nhiên lẫn Thuật toán giảm độ dốc theo lô nhỏ đều không đảm bảo sẽ có tiến triển trong mỗi lần lặp huấn luyện. Vì vậy, nếu bạn ngay lập tức dừng huấn luyện khi lỗi xác thực tăng lên, bạn có thể dừng quá sớm, trước khi đạt được điểm tối ưu. Một lựa chọn tốt hơn là lưu mô hình theo các khoảng thời gian đều đặn; sau đó, khi nó không cải thiện trong một thời gian dài (có nghĩa là nó có thể sẽ không bao giờ phá vỡ kỷ lục), bạn có thể quay lại mô hình đã lưu tốt nhất.
7. Thuật toán giảm độ dốc ngẫu nhiên có vòng lặp huấn luyện nhanh nhất vì nó chỉ xem xét một mẫu huấn luyện tại một thời điểm, vì vậy nó thường là thuật toán đầu tiên tiếp cận vùng lân cận của điểm tối ưu toàn cục (hoặc Thuật toán GD theo lô nhỏ với kích thước

- lô rất nhỏ). Tuy nhiên, chỉ có Thuật toán giảm độ dốc theo lô lớn mới thực sự hội tụ, nếu có đủ thời gian huấn luyện. Như đã đề cập, Thuật toán GD ngẫu nhiên và Thuật toán GD theo lô nhỏ sẽ dao động xung quanh điểm tối ưu, trừ khi bạn giảm tốc độ học dần dần.
8. Nếu lỗi xác thực cao hơn nhiều so với lỗi huấn luyện, điều này có thể là do mô hình của bạn đang quá khớp với tập huấn luyện. Một cách để khắc phục điều này là giảm bậc đa thức: một mô hình với ít bậc tự do hơn ít có khả năng quá khớp hơn. Một cách khác bạn có thể thử là điều chuẩn hóa (regularize) mô hình—ví dụ, bằng cách thêm một hình phạt ℓ_2 (Ridge) hoặc một hình phạt ℓ_1 (Lasso) vào hàm chi phí. Điều này cũng sẽ giảm bậc tự do của mô hình. Cuối cùng, bạn có thể thử tăng kích thước của tập huấn luyện.
 9. Nếu cả lỗi huấn luyện và lỗi xác thực gần như bằng nhau và khá cao, mô hình có khả năng bị dưới khớp (underfitting) với tập huấn luyện, có nghĩa là nó có độ chệch (bias) cao. Bạn nên thử giảm siêu tham số điều chuẩn α .
 10. Hãy xem xét: o Một mô hình có điều chuẩn thường hoạt động tốt hơn một mô hình không có điều chuẩn, vì vậy bạn nên ưu tiên Hồi quy Ridge (Ridge Regression) hơn Hồi quy Tuyến tính thông thường. o Hồi quy Lasso (Lasso Regression) sử dụng hình phạt ℓ_1 , có xu hướng đẩy các trọng số xuống chính xác bằng 0. Điều này dẫn đến các mô hình thưa thớt, trong đó tất cả các trọng số đều bằng 0 ngoại trừ các trọng số quan trọng nhất. Đây là một cách để thực hiện chọn lọc đặc trưng (feature selection) tự động, điều này tốt nếu bạn nghi ngờ rằng chỉ một vài đặc trưng thực sự quan trọng. Khi bạn không chắc chắn, bạn nên ưu tiên Hồi quy Ridge. o Elastic Net thường được ưu tiên hơn Lasso vì Lasso có thể hoạt động thất thường trong một số trường hợp (khi một vài đặc trưng có tương quan chặt chẽ hoặc khi có nhiều đặc trưng hơn số mẫu huấn luyện). Tuy nhiên, nó thêm một siêu tham số bổ sung để điều chỉnh. Nếu bạn muốn Lasso mà không có hành vi thất thường, bạn có thể sử dụng Elastic Net với `l1_ratio` gần bằng 1.
 11. Nếu bạn muốn phân loại hình ảnh là ngoài trời/trong nhà và ban ngày/ban đêm, vì đây không phải là các lớp loại trừ nhau (tức là cả bốn kết hợp đều có thể xảy ra) bạn nên huấn luyện hai bộ phân loại Hồi quy Logistic.

✓ 12. Batch Gradient Descent với việc dừng sớm cho Hồi quy Softmax

Bài tập: *Thực hiện Batch Gradient Descent với việc dừng sớm cho Hồi quy Softmax mà không cần sử dụng Scikit-Learn, chỉ sử dụng NumPy. Sử dụng nó trên một tác vụ phân loại như tập dữ liệu mống mắt.*

Hãy bắt đầu bằng cách tải dữ liệu. Chúng ta sẽ chỉ sử dụng lại tập dữ liệu Iris mà chúng ta đã tải trước đó.

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"].values
```

Chúng ta cần thêm thuật ngữ thiên vị cho mọi trường hợp ($x_0=1$). Tùy chọn dễ nhất để làm điều này là sử dụng Scikit-Learn's `add_dummy_feature()` chức năng, nhưng mục đích của bài tập này là hiểu rõ hơn về các thuật toán bằng cách triển khai chúng theo cách thủ công. Vì vậy, đây là một cách triển khai khả thi:

```
X_with_bias = np.c_[np.ones(len(X)), X]
```

Tùy chọn dễ nhất để chia tập dữ liệu thành một tập huấn luyện, một tập hợp xác thực và một tập thử nghiệm sẽ là sử dụng hàm của Scikit-Learn, nhưng một lần nữa, chúng ta muốn làm điều đó theo cách thủ công: `train_test_split()`

```
test_ratio = 0.2
validation_ratio = 0.2
total_size = len(X_with_bias)

test_size = int(total_size * test_ratio)
validation_size = int(total_size * validation_ratio)
train_size = total_size - test_size - validation_size

np.random.seed(42)
rnd_indices = np.random.permutation(total_size)

X_train = X_with_bias[rnd_indices[:train_size]]
y_train = y[rnd_indices[:train_size]]
X_valid = X_with_bias[rnd_indices[train_size:-test_size]]
y_valid = y[rnd_indices[train_size:-test_size]]
X_test = X_with_bias[rnd_indices[-test_size:]]
y_test = y[rnd_indices[-test_size:]]
```

Các mục tiêu hiện tại là các chỉ số lớp (0, 1 hoặc 2), nhưng chúng ta cần xác suất lớp mục tiêu để đào tạo mô hình Hồi quy Softmax. Mỗi trường hợp sẽ có xác suất lớp mục tiêu bằng 0,0 cho tất cả các lớp ngoại trừ lớp mục tiêu sẽ có xác suất là 1,0 (nói cách khác, vector xác suất lớp cho bất kỳ trường hợp nhất định nào là một vector một nóng). Hãy viết một hàm nhỏ để chuyển đổi vector của các chỉ số lớp thành một ma trận chứa một vector một nóng cho mỗi trường hợp. Để hiểu mã này, bạn cần biết rằng tạo ra một ma trận $n \times n$ đầy đủ các số 0

ngoại trừ 1 giây trên đường chéo chính. Hơn nữa, nếu là một mảng NumPy, thì trả về một mảng có 3 hàng bằng , và (đây là lập chỉ mục NumPy nâng cao [link text](#)).np.diag(np.ones(n))aa[[1, 3, 2]]a[1]a[3]a[2]

```
def to_one_hot(y):
    return np.diag(np.ones(y.max() + 1))[y]
```

Hãy kiểm tra hàm này trên 10 trường hợp đầu tiên:

```
y_train[:10]
```

```
array([1, 0, 2, 1, 1, 0, 1, 2, 1, 1])
```

```
to_one_hot(y_train[:10])
```

```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 1., 0.]])
```

Có vẻ ổn, vì vậy hãy tạo ma trận xác suất lớp mục tiêu cho tập huấn luyện và tập kiểm tra:

```
Y_train_one_hot = to_one_hot(y_train)
Y_valid_one_hot = to_one_hot(y_valid)
Y_test_one_hot = to_one_hot(y_test)
```

Bây giờ hãy mở rộng các đầu vào. Chúng tôi tính toán giá trị trung bình và độ lệch chuẩn của từng đối tượng địa lý trên tập huấn luyện (ngoại trừ tính năng thiên vị), sau đó chúng tôi căn giữa và chia tỷ lệ từng tính năng trong tập huấn luyện, tập hợp xác thực và tập thử nghiệm:

```
mean = X_train[:, 1:].mean(axis=0)
std = X_train[:, 1:].std(axis=0)
X_train[:, 1:] = (X_train[:, 1:] - mean) / std
X_valid[:, 1:] = (X_valid[:, 1:] - mean) / std
X_test[:, 1:] = (X_test[:, 1:] - mean) / std
```

Bây giờ chúng ta hãy triển khai hàm Softmax. Nhớ lại rằng nó được định nghĩa bởi phương trình sau:

$$\sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

```
def softmax(logits):
    exps = np.exp(logits)
    exp_sums = exps.sum(axis=1, keepdims=True)
    return exps / exp_sums
```

Chúng tôi gần như đã sẵn sàng để bắt đầu tập luyện. Hãy xác định số lượng đầu vào và đầu ra:

```
n_inputs = X_train.shape[1] # == 3 (2 features plus the bias term)
n_outputs = len(np.unique(y_train)) # == 3 (there are 3 iris classes)
```

Bây giờ đây là phần khó nhất: đào tạo! Về mặt lý thuyết, nó rất đơn giản: nó chỉ là vấn đề dịch các phương trình toán học thành mã Python. Nhưng trong thực tế, nó có thể khá phức tạp: đặc biệt, rất dễ trộn lẫn thứ tự của các thuật ngữ hoặc các chỉ số. Bạn thậm chí có thể kết thúc với mã trông giống như nó đang hoạt động nhưng thực sự không tính toán chính xác. Khi không chắc chắn, bạn nên viết ra hình dạng của từng thuật ngữ trong phương trình và đảm bảo các thuật ngữ tương ứng trong mã của bạn khớp chặt chẽ. Nó cũng có thể giúp đánh giá từng thuật ngữ một cách độc lập và in chúng ra. Tin tốt là bạn sẽ không phải làm điều này hàng ngày, vì tất cả những điều này đều được Scikit-Learn thực hiện tốt, nhưng nó sẽ giúp bạn hiểu những gì đang diễn ra bên dưới.

Vì vậy, các phương trình chúng ta sẽ cần là hàm chi phí:

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

Và phương trình cho các gradient:

$$\nabla_{\theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

Lưu ý rằng $\log(\hat{p}_k^{(i)})$ có thể không tính toán được nếu $\hat{p}_k^{(i)} = 0$. Vì vậy, chúng ta sẽ thêm một giá trị nhỏ ϵ vào $\log(\hat{p}_k^{(i)})$ để không nhận giá trị `nan`.

```
eta = 0.5
n_epochs = 5001
m = len(X_train)
epsilon = 1e-5

np.random.seed(42)
Theta = np.random.randn(n_inputs, n_outputs)

for epoch in range(n_epochs):
    logits = X_train @ Theta
    Y_proba = softmax(logits)
    if epoch % 1000 == 0:
        Y_proba_valid = softmax(X_valid @ Theta)
        xentropy_losses = -(Y_valid_one_hot * np.log(Y_proba_valid + epsilon))
        print(epoch, xentropy_losses.sum(axis=1).mean())
    error = Y_proba - Y_train_one_hot
    gradients = 1 / m * X_train.T @ error
    Theta = Theta - eta * gradients

0 3.7085808486476917
1000 0.14519367480830644
2000 0.1301309575504088
3000 0.12009639326384539
4000 0.11372961364786884
5000 0.11002459532472425
```

Và thế là xong! Mô hình Softmax được đào tạo. Hãy xem xét các thông số của mô hình:

Theta
array([[0.41931626, 6.11112089, -5.52429876], [-6.53054533, -0.74608616, 8.33137102], [-5.28115784, 0.25152675, 6.90680425]])

Hãy đưa ra dự đoán cho bộ xác thực và kiểm tra điểm chính xác:

```
logits = X_valid @ Theta
Y_proba = softmax(logits)
y_predict = Y_proba.argmax(axis=1)

accuracy_score = (y_predict == y_valid).mean()
accuracy_score

0.9333333333333333
```

Vậy, mô hình này trông khá ổn. Vì lợi ích của bài tập, chúng ta hãy thêm một chút ℓ_2 chính quy hóa. Mã đào tạo sau đây tương tự như mã trên, nhưng mất bây giờ có thêm ℓ_2 và các gradient có số hạng bổ sung thích hợp (lưu ý rằng chúng ta không quy tắc hóa phần tử đầu tiên của Theta vì điều này tương ứng với thuật ngữ thiên vị). Ngoài ra, chúng ta hãy thử tăng tỷ lệ học tập `.eta`

```
eta = 0.5
n_epochs = 5001
m = len(X_train)
epsilon = 1e-5
alpha = 0.01 # regularization hyperparameter

np.random.seed(42)
Theta = np.random.randn(n_inputs, n_outputs)

for epoch in range(n_epochs):
    logits = X_train @ Theta
```

```

Y_proba = softmax(logits)
if epoch % 1000 == 0:
    Y_proba_valid = softmax(X_valid @ Theta)
    xentropy_losses = -(Y_valid_one_hot * np.log(Y_proba_valid + epsilon))
    l2_loss = 1 / 2 * (Theta[1:] ** 2).sum()
    total_loss = xentropy_losses.sum(axis=1).mean() + alpha * l2_loss
    print(epoch, total_loss.round(4))
error = Y_proba - Y_train_one_hot
gradients = 1 / m * X_train.T @ error
gradients += np.r_[np.zeros([1, n_outputs]), alpha * Theta[1:]]
Theta = Theta - eta * gradients

```

```

0 3.7372
1000 0.3259
2000 0.3259
3000 0.3259
4000 0.3259
5000 0.3259

```

Vì bổ sung ℓ_2 phạt, tổn thất có vẻ lớn hơn trước đó, nhưng có lẽ mô hình này sẽ hoạt động tốt hơn? Hãy cùng tìm hiểu:

```

logits = X_valid @ Theta
Y_proba = softmax(logits)
y_predict = Y_proba.argmax(axis=1)

accuracy_score = (y_predict == y_valid).mean()
accuracy_score

```

```
0.9333333333333333
```

Trong trường hợp này, ℓ_2 hình phạt không làm thay đổi độ chính xác của bài kiểm tra. Có lẽ hãy thử tinh chỉnh alpha?

Bây giờ chúng ta hãy thêm điểm dừng sớm. Đối với điều này, chúng ta chỉ cần đo lường tổn thất trên xác thực được đặt ở mỗi lần lặp lại và dừng lại khi lỗi bắt đầu phát triển.

```

eta = 0.5
n_epochs = 50_001
m = len(X_train)
epsilon = 1e-5
C = 100 # regularization hyperparameter
best_loss = np.infty

np.random.seed(42)
Theta = np.random.randn(n_inputs, n_outputs)

for epoch in range(n_epochs):
    logits = X_train @ Theta
    Y_proba = softmax(logits)
    Y_proba_valid = softmax(X_valid @ Theta)
    xentropy_losses = -(Y_valid_one_hot * np.log(Y_proba_valid + epsilon))
    l2_loss = 1 / 2 * (Theta[1:] ** 2).sum()
    total_loss = xentropy_losses.sum(axis=1).mean() + 1 / C * l2_loss
    if epoch % 1000 == 0:
        print(epoch, total_loss.round(4))
    if total_loss < best_loss:
        best_loss = total_loss
    else:
        print(epoch - 1, best_loss.round(4))
        print(epoch, total_loss.round(4), "early stopping!")
        break
    error = Y_proba - Y_train_one_hot
    gradients = 1 / m * X_train.T @ error
    gradients += np.r_[np.zeros([1, n_outputs]), 1 / C * Theta[1:]]
    Theta = Theta - eta * gradients

```

```

0 3.7372
281 0.3256
282 0.3256 early stopping!

```

```

logits = X_valid @ Theta
Y_proba = softmax(logits)
y_predict = Y_proba.argmax(axis=1)

```

```
accuracy_score = (y_predict == y_valid).mean()
```

```
0.9333333333333333
```

Ồ tốt, vẫn không có thay đổi về độ chính xác thực, nhưng ít nhất là dừng sớm đã rút ngắn quá trình đào tạo một chút.

Bây giờ hãy vẽ các dự đoán của mô hình trên toàn bộ tập dữ liệu (nhớ chia tỷ lệ tất cả các tính năng được cung cấp cho mô hình):

```
custom_cmap = mpl.colors.ListedColormap(['#fafab0', '#9898ff', '#a0faa0'])

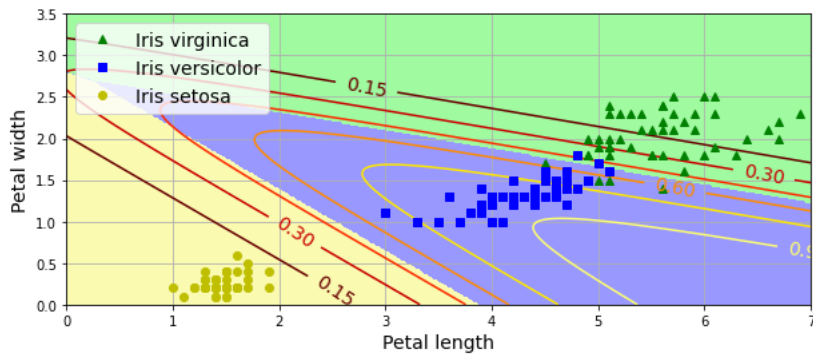
x0, x1 = np.meshgrid(np.linspace(0, 8, 500).reshape(-1, 1),
                     np.linspace(0, 3.5, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()]
X_new = (X_new - mean) / std
X_new_with_bias = np.c_[np.ones(len(X_new)), X_new]

logits = X_new_with_bias @ Theta
Y_proba = softmax(logits)
y_predict = Y_proba.argmax(axis=1)

zz1 = Y_proba[:, 1].reshape(x0.shape)
zz = y_predict.reshape(x0.shape)

plt.figure(figsize=(10, 4))
plt.plot(X[y == 2, 0], X[y == 2, 1], "g^", label="Iris virginica")
plt.plot(X[y == 1, 0], X[y == 1, 1], "bs", label="Iris versicolor")
plt.plot(X[y == 0, 0], X[y == 0, 1], "yo", label="Iris setosa")

plt.contourf(x0, x1, zz, cmap=custom_cmap)
contour = plt.contour(x0, x1, zz1, cmap="hot")
plt.clabel(contour, inline=1)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="upper left")
plt.axis([0, 7, 0, 3.5])
plt.grid()
plt.show()
```



Và bây giờ chúng ta hãy đo độ chính xác của mô hình cuối cùng trên tập thử nghiệm:

```
logits = X_test @ Theta
Y_proba = softmax(logits)
y_predict = Y_proba.argmax(axis=1)

accuracy_score = (y_predict == y_test).mean()
accuracy_score
```

```
0.9666666666666667
```

Vâng, chúng ta nhận được hiệu suất thậm chí còn tốt hơn trên bộ thử nghiệm. Sự thay đổi này có thể là do kích thước rất nhỏ của tập dữ